

Qué hay que implementar — Resumen simple

Este documento resume, en lenguaje simple, qué le falta al sistema de tracking de vulnerabilidades de GEF_Secure_App para ser considerado sólido. Es la versión corta de Brechas_Tracking_Remediacion.md — ahí está el detalle técnico completo, con archivos y líneas de código citadas; acá está el resumen para entender rápido qué hacer y por qué importa.

En una frase: hoy el sistema registra vulnerabilidades y las mueve por un tablero, pero no puede demostrar que las corrigió de verdad, no guarda quién cambió qué ni cuándo, y algunos de los números que muestra el dashboard no significan exactamente lo que dicen significar.

La buena noticia primero: varias cosas ya están mejor resueltas de lo que parecía a primera vista — el cálculo de prioridad ya combina varias señales (no es solo un puntaje de severidad), el responsable ya se elige de una lista de usuarios reales en pantalla, y el sistema ya conserva el historial de una vulnerabilidad en vez de borrarla cuando se reabre. El trabajo que falta es completar y conectar piezas, no empezar de cero.

Lo más urgente — sin esto, nada de lo demás es confiable

1. Guardar un historial de cada cambio, no solo el estado actual. Hoy, si alguien mueve una vulnerabilidad de "en análisis" a "resuelta", el sistema solo actualiza el estado — no queda ningún registro de que ese cambio puntual ocurrió, cuándo, ni quién lo hizo. Es como borrar el pizarrón cada vez que se anota algo nuevo. Sin este historial, ninguna de las mejoras siguientes se puede auditar de verdad más adelante.

2. Antes de cerrar una vulnerabilidad porque "ya no aparece" en el escaneo siguiente, chequear que el escaneo se haya hecho bien. Hoy, si un escaneo sale mal a medias, el sistema igual da por resueltas las vulnerabilidades que no volvió a ver — aunque en realidad nunca se volvieron a mirar. Es el error más peligroso que puede tener un sistema de este tipo, porque hace que las métricas mejoren justo cuando la vigilancia empeora, sin que nadie se dé cuenta. Buena noticia: el sistema **ya tiene** el dato de que un escaneo salió mal parcialmente — solo falta que lo consulte antes de cerrar.

3. Mostrar cuántas veces reapareció algo que ya se había dado por cerrado. El dato ya se calcula puertas adentro, pero hoy no se muestra en ningún lado — ni en el dashboard, ni en ningún reporte. Es la brecha más barata de cerrar de toda la lista: es información que ya existe, solo falta exponerla.

Importante — mejora directamente lo que se mide y se decide

4. Separar "cuánto tardamos en marcarlo resuelto" de "cuánto tardamos en confirmar que se arregló de verdad". Hoy el dashboard muestra un número que llama "MTTR" (tiempo de resolución), pero en realidad mide el tiempo hasta que una persona lo marcó como resuelto — no hasta que el sistema comprobó nada. Hay que separar esos dos números y llamarlos por lo que realmente son. De paso, conviene agregar dos gráficos que salen casi gratis con el mismo dato: antigüedad de lo que sigue abierto, y tiempo de resolución según la criticidad.

5. Agregar más formas de cerrar un caso, además de "resuelta". Hoy solo existe "detectada", "en análisis" y "resuelta". Falta poder marcar: "mitigada" (se redujo el riesgo sin eliminar la causa raíz), "no aplica" (con una justificación obligatoria, no opcional) y "riesgo aceptado" (con una fecha de vencimiento obligatoria, para que no quede aceptado para siempre sin que nadie lo revise).

6. Guardar por qué el sistema calculó tal prioridad, no solo el resultado final. El cálculo de prioridad ya es bastante bueno — ya combina la severidad de la vulnerabilidad, qué tan crítico es el entorno donde vive, si está en la lista de vulnerabilidades que el gobierno de EE. UU. confirma que se están explotando activamente, y si es tan nueva que todavía no tiene parche. El problema es que el sistema solo guarda el resultado final ("CRÍTICA"), no el detalle de por qué llegó a esa conclusión — y sin ese detalle, nadie puede auditar después una decisión de prioridad.

7. Agregar una fecha límite para resolver cada caso. Hoy no existe ningún plazo. Con el cálculo del punto 6 ya resuelto, convertirlo en una fecha concreta es relativamente simple.

Conviene hacerlo, pero es menos urgente

8. Poder calificar qué tan fuerte es la prueba de que algo se resolvió, y guardar esa prueba. Hoy no hay ninguna forma de distinguir "alguien dijo que lo arregló" de "hay un reporte de escaneo, un commit o un PR que lo confirma" — y tampoco hay dónde adjuntar o

linkear esa prueba, ni en el sistema ni en la pantalla.

9. Mejorar los datos que se traen de las vulnerabilidades de código abierto, y usarlos para verificar de verdad. Hoy el sistema no guarda el rango exacto de versiones afectadas de cada vulnerabilidad, ni si el aviso fue revisado por una persona o generado automáticamente, ni si fue retirado después de publicado. Con esos datos, se puede construir la comparación real: "¿la versión instalada hoy sigue estando en el rango afectado?" — en vez de asumir que, si algo desapareció del escaneo, se arregló. **Este es, técnicamente, el cambio más importante de toda la lista:** es el que hace que el sistema pase de "confiar en que se arregló" a "comprobar que se arregló". Sin este punto, ningún otro cambio alcanza para decir que el sistema verifica de verdad.

10. Que el responsable asignado sea una referencia real a un usuario, no solo un texto. En la pantalla ya se elige de una lista de usuarios reales — falta que la base de datos lo trate igual (hoy guarda solo el nombre como texto suelto).

11. Un campo para marcar si un activo está expuesto a internet. Es opcional y de menor prioridad — el punto 6 ya cubre buena parte de esta necesidad usando la criticidad del entorno.

¿Y si se hace todo esto?

El sistema deja de ser un tablero de tareas con fechas y pasa a ser un tracking de remediación real, en el sentido que le da la literatura de ciberseguridad: **comprueba por sí mismo que las cosas se arreglaron, en vez de creerle a quien lo dice.** Ese es el salto que importa, y el punto 9 es el que efectivamente lo cierra — hacer todo lo demás sin ese punto deja al sistema mejor organizado, pero todavía declarativo en el fondo.